

A Class for Status and Optional Value

Committee: ISO/IEC JTC1 SC22 WG21 EWG Evolution

Document Number: P0262r0

Date: 2016-02-14

Authors: Lawrence Crowl, Chris Mysisen

Reply To: Lawrence Crowl, Lawrence@Crowl.org

[Introduction](#)

[Discussion](#)

[Solution](#)

[Examples](#)

[Revision History](#)

Introduction

The current proposal for concurrent queues, [P0260r0 C++ Concurrent Queues](#), has two separate waiting pop operations, one with a return value one with an output reference parameter. These functions provide essentially the same semantics, but differ how they signal disappointment. See [P0157r0 Handling Disappointment in C++](#).

```
Value queue::value_pop();
queue_op_status queue::wait_pop(Value&);
```

This design has two consequences.

- A bad status for `value_pop` will result in an exception.
- Use of the `wait_pop` operation requires a *Value* that is essentially default constructible.

Chandler Carruth suggested having a return value that comprised both status and value.

```
something<queue_op_status, Value> queue::pop();
```

The essential point of the design is that *something* may or may not contain a value, and that accessing a non-existent value results in an exception.

This design would normalize the form of the functions, move exception generation to accessing a non-existent value, and enable use of types with no default constructor as elements.

This paper proposes a mechanism to provide this combined status and value. While such a proposal could be folded into the queue proposal alone, the true value of such a mechanism would be its widespread use throughout the library, particularly in concurrent data structures where it is not technically possible to provide separate access functions for status and value. Therefore, we should either commit to or abandon such a mechanism.

Discussion

The value is clearly related to the class template `optional`, as defined by [N3793 A proposal to add a utility class to represent optional objects \(Revision 5\)](#). The problem could be solved with a tuple of the status and an `optional`. That approach would require more verbose code at each use, which we prefer to avoid.

However, the need is almost directly met by the class template `expected`, as defined by [N4109 A proposal to add a utility class to represent expected monad - Revision 1](#).

In our view, the weakness in the `expected` proposal is that you get either a value or an error, but not both. In the queue proposal, operations return a status, not an error. The distinction is that a queue's inability to deliver a value is often not an error, but simply a normal part of interacting with concurrent objects. For example, `try_pop` can return `queue_op_status::empty`, which clearly does not indicate an error.

The distinction has a design effect when more than one status may be associated with a value. For example, a concurrent queue could provide both a status indicating success with low contention and a status indicating success with high contention. In both cases, the value alone provides less information.

So, we prefer a design in which a status is always provided, but the value is optional. Users of the design will need to define which statuses have values.

Solution

The design is a simpler version of that which appears in [N4109 A proposal to add a utility class to represent expected monad - Revision 1](#).

The working name for the proposed class is `status_value`.

```
template<typename Status, typename Value> class status_value;
```

Construction of a `status_value` can be done with or without a value.

```
status_value::status_value(Status s);
status_value::status_value(Status s, Value&& v);
status_value::status_value(Status s, const Value& v);
```

Construction of `status_value` must include a status.

```
status_value::status_value() = delete;
```

A `status_value` may be moved. A copy operation would make the type unusable for non-copyable contained objects, so we do not provide a copy operation.

```
status_value::status_value(status_value&& sv);
```

They may be queried for status. The design assumes that inlining will remove the cost of returning a reference for cheap copyable types.

```
const Status& status_value::status() const;
```

They may be queried for whether or not they have a value.

```
bool status_value::has_value() const;
status_value::operator bool() const;
```

They may provide access to their value. If they have no value, an exception of type `Status`, with the status value passed to the constructor, is thrown.

```
const Value& status_value::value() const;
Value& status_value::value();
const Value& status_value::operator *() const;
Value& status_value::operator *();
```

This design enables moving out of the class by calling `std::move` on the result of the non-const functions.

Examples

The outlined solution changes typical code for the proposed concurrent queue from

```
Value e = q.value_pop();
```

into

```
Value e = q.value_pop().value();
```

and from

```
Value e;
queue_op_status s = q.wait_pop(e);
if ( s == queue_op_status::success )
    do_something_with(e);
else
    do_something_else_with(s);
```

into

```
auto sv = q.wait_pop();
if ( sv.status() == queue_op_status::success )
    do_something_with(sv.value());
else
    do_something_else_with(sv.status());
```

or for handling all statuses that have values

```
if ( auto sv = q.wait_pop() )
```

```
    // Via the implicit conversion to bool, a value is known to be present.  
    do_something_with(*sv);  
else  
    // Via the implicit conversion to bool, a value is known to be absent.  
    do_something_else_with(sv.status());
```

With both sets of changes, the handling of disappointment is decided by the caller, not by the set of operations provided by the queue.

Revision History

This paper revises N4233 - 2014-10-10 as follows.

- Add a reference to [P0157r0 Handling Disappointment in C++](#).
- Add a discussion of a tuple of status and optional.
- Explicitly avoid copying for `status_value` objects.
- Make the `status` member function return a reference, which enables efficient access to non-trivial status objects.
- Clarify the mechanism for moving out of a `status_value`.
- Change the running example to `value_pop` versus `wait_pop`, which is the essential example of different handling of disappointment.
- Add clarification of the example using implicit conversion to `bool` in a variable declaration in a condition.